Scenario 5: High Application Latency

Objective

Measure latency from outside the cluster, isolate the slow workload, then investigate Kubernetes and apply a focused recovery.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/inject-latency.sh ecommerce 3000
```

The request reaches one backend pod because chaos state is stored per process. Only requests served by that pod are slow, so the symptom is intermittent. To affect every backend pod, use the port-forward method under Fix to send the same request to each pod.

Situation

Users report that ecommerce pages load slowly. Do not restart pods before scoping the problem.

Symptoms

- Ecommerce responses are intermittently slow.
- Pods remain Ready.

What You Know

- The issue affects `ecommerce-backend` while pods remain Ready.
- The start time does not match an AWS or database change.

Start With the Symptom

1. Time repeated API requests from outside the cluster:

```
for i in {1..10}; do
  curl -s -o /dev/null -w '%{http_code} %{time_total}s\n' \
    "https://ecommerce.$(cat .lab-domain)/api/products"
done
```

Expected output: every request returns `200`, but some take about three seconds while the rest are fast.

1. Time the static frontend the same way:

```
curl -s -o /dev/null -w '%{http_code} %{time_total}s\n' "https://ecommerce.$(cat .lab-domain)/"
```

Expected output: consistently fast. DNS, the ALB, and the frontend are fine, so the delay is in the backend.

Record the start time, the slow and fast response times, and how many requests were slow.

Troubleshooting

1. Confirm Kubernetes health.

```
kubectl get deployment,pods -n ecommerce
```

Expected output: the Deployment is fully available, pods are Ready, and restart counts remain stable. The application is slow but not unavailable.

1. Check resource usage.

```
kubectl top pods -n ecommerce
```

Expected output: CPU and memory remain near their normal baseline. This makes resource saturation and OOM restarts unlikely.

1. Time each backend pod directly to find the slow one.

```
kubectl get pods -n ecommerce -l app=ecommerce-backend
kubectl port-forward -n ecommerce pod/<pod-name> 4000:4000
# in a second terminal:
curl -s -o /dev/null -w '%{time_total}s\n' http://localhost:4000/api/products
```

Expected output: one pod answers in about three seconds and the others answer quickly. The slowdown lives inside one process, not in the network path.

1. Check that pod's chaos state through the same port-forward.

```
curl -s http://localhost:4000/api/chaos
```

Expected output: JSON containing `"latencyMs":3000`. This in-memory setting explains requests taking about three seconds while Kubernetes remains healthy.

Important Clues

Determine why one or more backend containers add time before every response. Use per-pod response times and the chaos-state endpoint as evidence.

Root Cause

Record the workload behavior that accounts for the response time while Kubernetes resource metrics remain normal, before you apply the fix.

Fix

Clear the injected latency. Chaos state is per process, so reset every backend pod. The simplest way is through a port-forward to each pod:

```
kubectl port-forward -n ecommerce pod/<pod-name> 4000:4000
# in a second terminal:
curl -s -X POST http://localhost:4000/api/chaos/reset
```

`./scripts/chaos/reset.sh ecommerce` does the same through the public URL, but it only reaches whichever pod the load balancer picks, so repeat it and re-check each pod.

Validation

Repeat the timing loop from Start With the Symptom. Confirm every request is fast again, every pod reports `"latencyMs":0`, and pods remain healthy.

DevOps Lesson

Measure and isolate the slow workload before taking action. A restart may hide evidence without explaining the cause.

STAR Method

Situation

Summarize the latency report and initial scope.

Task

Explain the need to identify the affected workload before acting.

Action

Describe the external timing, per-pod timing, Kubernetes, and workload-state evidence.

Result

State how normal latency and healthy pods were verified.

Natural Spoken Version

Use the matching example in DevOps STAR Scenarios after completing the lab.

Brief STAR Example

Users reported slow responses from one of our applications while its EKS pods remained healthy. My task was to isolate the source before restarting anything. Timing requests showed API calls taking about three seconds on one backend pod, while CPU, memory, readiness, and restarts stayed normal. I found and cleared a 3000 ms application delay setting on that pod, after which response times returned to baseline.